4.4.4 Binary Search Tree: Every Operation is Logarithmic

BSTs are another old acquaintance of ours, we have met them in chapter 2, and they as well are introduced in appendix C.

A BST is a special kind of binary tree that can store keys on which it is defined a total ordering: this means that for each pair of keys, it must be possible to compare them and decide which one is smaller, or if they are equal. A total ordering benefits from *reflexive*, *symmetric* and *transitive* properties.

Given a set S on which we defined an ordering relation ≤, this relation is a total ordering iff, for any three keys x, y, z, the following properties hold:
Reflexive: $x \leq x$
Symmetric: if $x \leq y$, then $y \leq x$
Transitive: if $x \leq y$ and $y \leq z$ then $x \leq z$

BSTs use these properties to make sure that the position of a key in the tree can be determined just by looking at a single path from the root to a leaf.

When we insert a new key, in fact, we compare it to the tree's root: if it's smaller we "take a left turn", traversing the root's left subtree, otherwise we go on the right subtree. At the next step, we repeat the comparison with the subtree's